

ML From Scratch Project Roadmap

This roadmap turns the project documentation into a practical sequence of work. Its purpose is to help you always know:

- ? What to build next.
- ? Why that step matters.
- ? What skills you are practicing.
- ? What "done" means before moving on.

The project should grow in layers: first structure, then correctness, then reusable infrastructure, then algorithms, then benchmarking, then C optimization.

Guiding Rule

Do not optimize before the simple version works and is tested.

For every major component, follow this order:

1. Understand the math.
2. Implement a clear Python version.
3. Add tests.
4. Validate behavior on small synthetic data.
5. Benchmark.
6. Identify bottlenecks.
7. Move hot paths to C only when the need is clear.
8. Re-test and re-benchmark.

Phase 0: Project Foundation

Goal: Create a clean project skeleton that feels like a real library from day one.

Tasks

- ? Create the main repository structure:
 - src/
 - src/linear_models/
 - src/preprocessing/
 - src/metrics/

- src/model_selection/
- src/utils/
- csrc/
- tests/
- benchmarks/
- docs/
- experiments/
- scripts/
- ? Add README.md.
- ? Add a simple project overview in README.md.
- ? Add a docs/architecture/ folder.
- ? Add docs/algorithms/ folder.
- ? Add docs/optimization-notes/ folder.
- ? Decide on a package name, for example mlscratch.
- ? Create package initialization files.
- ? Add a basic test runner setup.

Recommended First Files

```
src/mlscratch/__init__.py
src/mlscratch/linear_models/__init__.py
src/mlscratch/preprocessing/__init__.py
src/mlscratch/metrics/__init__.py
src/mlscratch/model_selection/__init__.py
src/mlscratch/utils/__init__.py
tests/test_project_imports.py
README.md
```

Definition of Done

- ? The project has a recognizable library structure.
- ? You can import the package from a simple test.
- ? The README explains the project mission.
- ? There is no algorithm code yet unless needed for import testing.

Phase 1: Core Python Utilities

Goal: Build the shared tools that every algorithm will depend on.

This phase matters because a library becomes messy quickly if every model handles input checking, shapes, metrics, and data splitting differently.

1.1 Data Validation

Build utilities for checking and normalizing input data.

Suggested file:

```
src/mlscratch/utils/validation.py
```

Functions to implement:

? `check_X_y(X, y)`

? `check_X(X)`

? `check_vector(y)`

? `ensure_2d(X)`

? `num_samples(X)`

? `num_features(X)`

Rules:

? Accept Python lists and tuples.

? Use only Python standard library in core implementation.

? Avoid NumPy in core code.

? Make errors clear and helpful.

Definition of done:

? Handles valid 2D feature matrices.

? Rejects empty data.

? Rejects inconsistent row lengths.

? Rejects mismatched X and y lengths.

? Tests cover normal and broken inputs.

1.2 Basic Data Containers

Goal: Decide how dense data is represented internally.

Suggested file:

```
src/mlscratch/utils/arrays.py
```

Start simple:

? Use lists for first prototypes.

? Add conversion to `array('d')` later when C kernels become relevant.

Functions to implement:

? `to_float_matrix(X)`

? `to_float_vector(y)`

? `zeros(length)`

? `dot(a, b)`

? `mean(values)`

Definition of done:

? Algorithms can rely on these helpers instead of rewriting conversions.

? Tests cover numeric conversion and dot products.

1.3 Metrics

Suggested files:

```
src/mlscratch/metrics/regression.py
src/mlscratch/metrics/classification.py
```

Regression metrics:

? Mean Squared Error

? Root Mean Squared Error

? Mean Absolute Error

? R2 Score

Classification metrics:

? Accuracy

? Precision

? Recall

? F1 Score

? Confusion Matrix

Definition of done:

? Metrics work on plain Python lists.

? Edge cases are handled.

? Tests include small hand-computed examples.

1.4 Model Selection Basics

Suggested file:

```
src/mlscratch/model_selection/split.py
```

Functions:

```
? train_test_split(X, y, test_size=0.2, shuffle=True, random_state=None)
? Later: k_fold_split(X, y, n_splits=5, shuffle=True, random_state=None)
```

Definition of done:

- ? Splitting is reproducible with `random_state`.
- ? Data and labels stay aligned.
- ? Tests check sizes and reproducibility.

Phase 2: First Estimator API

Goal: Establish the interface every model should follow.

Suggested file:

```
src/mlscratch/base.py
```

Create simple base classes:

```
? BaseEstimator
? RegressorMixin
? ClassifierMixin
```

Expected estimator behavior:

```
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
score = model.score(X_test, y_test)
```

Definition of done:

- ? All future models have a consistent shape.
- ? `score()` uses the right default metric:
- Regression: R^2
- Classification: Accuracy
- ? Tests verify the interface.

Phase 3: Linear Regression

Goal: Build the first polished algorithm from start to finish.

This should become the model example for the rest of the library.

3.1 Linear Regression Math Prototype

Suggested file:

```
src/mlscratch/linear_models/linear_regression.py
```

Implement:

- ? Batch gradient descent.
- ? Optional intercept.
- ? Learning rate.
- ? Number of iterations.
- ? Loss history.
- ? `fit()`
- ? `predict()`
- ? `score()`

Initial class:

```
class LinearRegression:
    def __init__(self, learning_rate=0.01, n_iters=1000, fit_intercept=True):
        ...
```

Definition of done:

- ? Fits simple synthetic linear data.
- ? Loss decreases during training.
- ? Predictions are close to expected values.
- ? Tests cover one-feature and multi-feature input.

3.2 Analytical Linear Regression

Add an optional normal-equation implementation later.

Important constraint:

- ? Do not use NumPy linear algebra in core code.

Possible approaches:

- ? Implement small matrix operations yourself.
- ? Start with gradient descent only if matrix inversion feels like a distraction.

Definition of done:

- ? You understand the tradeoff between closed-form and iterative training.
- ? The implementation stays readable.

3.3 Linear Regression Documentation

Create:

```
docs/algorithms/linear_regression.md
```

Include:

- ? Problem definition.
- ? Loss function.
- ? Gradient formulas.
- ? Implementation notes.
- ? Known limitations.
- ? Test examples.

Phase 4: Preprocessing

Goal: Support the transformations needed by linear models and later algorithms.

Suggested files:

```
src/mlscratch/preprocessing/scalers.py  
src/mlscratch/preprocessing/features.py
```

Implement:

- ? StandardScaler
- ? MinMaxScaler
- ? PolynomialFeatures

Expected API:

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Definition of done:

- ? Scalers store fitted statistics.
- ? Calling `transform()` before `fit()` raises a clear error.
- ? Tests cover `fit`, `transform`, and `fit_transform`.
- ? Polynomial features work for small cases.

Phase 5: Regularized Linear Models

Goal: Extend linear regression into the main family of linear models.

Suggested file:

```
src/mlscratch/linear_models/regularized.py
```

Implement in this order:

1. Ridge Regression
2. Lasso Regression
3. Elastic Net
4. SGD Regression

Ridge Regression

Skills practiced:

- ? L2 regularization.
- ? Bias-variance tradeoff.
- ? Gradient modification.

Definition of done:

- ? Coefficients shrink compared with unregularized regression.
- ? Intercept is not regularized.
- ? Tests verify behavior on synthetic data.

Lasso Regression

Skills practiced:

- ? L1 regularization.
- ? Sparsity.
- ? Subgradients or coordinate descent.

Definition of done:

- ? Some coefficients can move toward zero.
- ? Tests verify basic convergence.

Elastic Net

Skills practiced:

- ? Combining L1 and L2.
- ? Hyperparameter design.

Definition of done:

- ? Behaves like Ridge when L1 ratio is 0.
- ? Behaves like Lasso when L1 ratio is 1.

SGD Regression

Skills practiced:

- ? Stochastic optimization.
- ? Epochs.
- ? Shuffling.
- ? Learning-rate schedules.

Definition of done:

- ? Can train on larger synthetic data.
- ? Supports reproducible shuffling.
- ? Loss trend is reasonable.

Phase 6: Logistic Regression and Classification Basics

Goal: Move from regression into classification.

Suggested file:

```
src/mlscratch/linear_models/logistic_regression.py
```

Implement:

- ? Binary logistic regression.

- ? Sigmoid function with numerical stability.
- ? Binary cross-entropy loss.
- ? Class prediction.
- ? Probability prediction.

Expected API:

```
model = LogisticRegression()
model.fit(X_train, y_train)
labels = model.predict(X_test)
probs = model.predict_proba(X_test)
```

Definition of done:

- ? Works on linearly separable synthetic data.
- ? Accuracy improves after training.
- ? Tests cover labels, probabilities, and threshold behavior.

Then implement:

- ? Softmax Regression
- ? SGD Classifier

Phase 7: Benchmarking Infrastructure

Goal: Make performance measurable before writing C.

Suggested files:

```
benchmarks/bench_linear_regression.py
benchmarks/bench_logistic_regression.py
src/mlscratch/utils/timing.py
```

Implement:

- ? Synthetic dataset generators.
- ? Timing helpers.
- ? Result logging.
- ? Single-threaded baseline measurements.

Benchmark dimensions:

- ? n = 1_000, 10_000, 100_000
- ? d = 10, 50, 200

Keep 1_000_000 samples for later once the library is stable.

Definition of done:

- ? You can run a benchmark and get repeatable timing.
- ? Benchmarks report dataset size, feature count, iterations, and time.
- ? Results are saved or easy to copy into docs.

Phase 8: C Kernel Foundation

Goal: Start the systems side of the project carefully.

Suggested folders:

```
csrc/linear_algebra/  
csrc/linear_models/  
scripts/
```

Start with small kernels:

- ? Dot product.
- ? Vector sum.
- ? Mean squared error.
- ? Linear regression prediction.

Suggested files:

```
csrc/linear_algebra/dot.c  
csrc/linear_algebra/dot.h  
src/mlscratch/utils/native.py  
scripts/build_native.ps1
```

Python integration:

- ? Use ctypes.
- ? Keep pure Python fallback behavior.

Definition of done:

- ? C dot product returns the same result as Python dot product.
- ? Tests compare C and Python outputs.
- ? Build script works on your system.
- ? The Python library still works if the C library is unavailable.

Phase 9: Linear Regression C Acceleration

Goal: Move the hottest linear regression operations into C.

Possible kernels:

- ? Batch prediction.
- ? Residual computation.
- ? Gradient accumulation.
- ? One training epoch.

Suggested C function:

```
int linreg_epoch_f64(  
    const double* X,  
    const double* y,  
    double* w,  
    double* b,  
    size_t n,  
    size_t d,  
    double lr  
);
```

Definition of done:

- ? C and Python implementations produce similar results.
- ? Tests cover correctness.
- ? Benchmarks show whether performance improved.
- ? Documentation records what changed and why.

Phase 10: Decision Trees

Goal: Implement the first non-linear model family.

Suggested files:

```
src/mlscratch/tree_models/decision_tree.py  
src/mlscratch/tree_models/criteria.py
```

Implement:

- ? Decision Tree Classifier first.
- ? Gini impurity.
- ? Entropy.
- ? Best split search.
- ? Max depth.
- ? Minimum samples split.
- ? Leaf prediction.

Then implement:

- ? Decision Tree Regressor.
- ? MSE split criterion.
- ? Optional pruning.

Definition of done:

- ? Classifier solves simple synthetic classification tasks.
- ? Regressor fits simple piecewise data.
- ? Tests cover split criteria and predictions.
- ? Tree structure is inspectable.

Phase 11: Ensemble Models

Goal: Use trees to build stronger models.

Implement in this order:

1. Bagging Classifier
2. Random Forest Classifier
3. Random Forest Regressor
4. Extremely Randomized Trees
5. AdaBoost
6. Gradient Boosting

Suggested folder:

```
src/mlscratch/ensemble/
```

Definition of done:

- ? Bootstrapping works.
- ? Feature subsampling works.
- ? Ensemble predictions combine correctly.
- ? Benchmarks compare single tree vs ensemble.

Phase 12: Support Vector Machines

Goal: Understand margin-based classification.

Suggested file:

`src/mlscratch/svm/svm.py`

Implement in this order:

1. Linear SVM with hinge loss.
2. Soft-margin classification.
3. Polynomial kernel.
4. RBF kernel.

Definition of done:

- ? Linear SVM works on separable data.
- ? Soft-margin handles noisy data.
- ? Kernel versions solve simple non-linear data.
- ? Documentation explains primal vs dual choices.

Phase 13: Clustering

Goal: Add unsupervised learning.

Suggested folder:

`src/mlscratch/clustering/`

Implement:

1. K-Means
2. Mini-Batch K-Means
3. DBSCAN
4. Gaussian Mixture Models as optional advanced work

Definition of done:

- ? K-Means converges on simple clusters.
- ? Mini-Batch K-Means works on larger synthetic datasets.
- ? DBSCAN identifies dense regions and noise.
- ? Distance calculations are benchmarked.

Phase 14: Dimensionality Reduction

Goal: Build numerical linear algebra understanding.

Suggested folder:

```
src/mlscratch/decomposition/
```

Implement:

1. PCA
2. Incremental PCA
3. Randomized PCA

Definition of done:

- ? PCA reduces dimensions on synthetic data.
- ? Explained variance is computed.
- ? Reconstruction error is measurable.
- ? Documentation explains the numerical method used.

Phase 15: Cross-Validation and Search

Goal: Build model evaluation workflows.

Suggested files:

```
src/mlscratch/model_selection/cross_validation.py  
src/mlscratch/model_selection/search.py
```

Implement:

- ? K-fold cross-validation.
- ? Stratified split for classification.
- ? Grid search.
- ? Random search.

Definition of done:

- ? Works with any estimator using the common API.
- ? Reports mean and per-fold scores.
- ? Tests use small dummy estimators.

Phase 16: Advanced C Optimization

Goal: Move from "C works" to "C is actually fast."

Optimization topics:

- ? Cache locality.
- ? Row-major traversal.
- ? Loop fusion.
- ? OpenMP parallel loops.
- ? SIMD-friendly data alignment.
- ? Compiler optimization flags.
- ? Branch reduction.

Possible targets:

- ? Matrix-vector multiplication.
- ? Distance kernels for K-Means.
- ? Histogram building for decision trees.
- ? Split scoring.
- ? Batch prediction.

Definition of done:

- ? Each optimization has a benchmark before and after.
- ? Each optimization has a short note in docs/optimization-notes/.
- ? Correctness tests still pass.

Phase 17: Polish, Packaging, and Portfolio Quality

Goal: Turn the project into something readable, runnable, and impressive.

Tasks:

- ? Improve README.
- ? Add examples.
- ? Add installation instructions.
- ? Add algorithm documentation.
- ? Add benchmark summaries.
- ? Add architecture notes.
- ? Add known limitations.
- ? Add contribution notes for yourself.

Suggested examples:

```
examples/linear_regression_demo.py
examples/logistic_regression_demo.py
```


examples/kmeans_demo.py
examples/random_forest_demo.py

Definition of done:

- ? A new reader can understand the project in five minutes.
- ? A user can run an example without guessing.
- ? The documentation explains both ML and systems decisions.

Recommended First 10 Concrete Steps

If you are starting today, follow these steps exactly:

1. Create the repository structure.
2. Create the mlscratch package.
3. Add a README.
4. Add input validation helpers.
5. Add regression metrics.
6. Add train_test_split.
7. Add base estimator classes.
8. Implement pure Python Linear Regression with gradient descent.
9. Add tests for Linear Regression.
10. Add a small benchmark for Linear Regression.

After those steps, pause and review:

- ? Is the API clean?
- ? Are tests easy to write?
- ? Are errors understandable?
- ? Is the code still readable?
- ? Do benchmarks reveal an obvious bottleneck?

Suggested Weekly Plan

This is a practical pacing guide. You can move faster or slower, but each week should end with something working.

Week 1: Structure and Utilities

- ? Create repo structure.

- ? Create package.
- ? Add validation helpers.
- ? Add basic metrics.
- ? Add train/test split.
- ? Add tests.

Outcome:

- ? The project feels like a real library.

Week 2: Linear Regression

- ? Implement Linear Regression.
- ? Add loss tracking.
- ? Add tests.
- ? Add documentation.
- ? Add simple benchmark.

Outcome:

- ? First complete model.

Week 3: Preprocessing

- ? Implement StandardScaler.
- ? Implement MinMaxScaler.
- ? Implement PolynomialFeatures.
- ? Add tests.
- ? Use preprocessing with Linear Regression.

Outcome:

- ? Linear models become more useful.

Week 4: Ridge, Lasso, Elastic Net

- ? Implement Ridge.
- ? Implement Lasso.
- ? Implement Elastic Net.
- ? Compare coefficient behavior.
- ? Document regularization.

Outcome:

- ? Core regression family exists.

Week 5: Logistic Regression

- ? Implement binary Logistic Regression.
- ? Add classification metrics.
- ? Add tests.
- ? Add simple classification benchmark.

Outcome:

- ? First classifier.

Week 6: SGD Models

- ? Implement SGD Regression.
- ? Implement SGD Classifier.
- ? Add shuffling and random state.
- ? Add learning-rate schedule.

Outcome:

- ? Library can handle larger datasets more naturally.

Week 7: Benchmarking and First C Kernels

- ? Build benchmark helpers.
- ? Implement C dot product.
- ? Connect C through ctypes.
- ? Compare Python vs C.

Outcome:

- ? First native kernel integration.

Week 8: Linear Regression Native Kernel

- ? Implement C prediction or epoch kernel.
- ? Add tests.
- ? Add benchmarks.
- ? Write optimization note.

Outcome:

? First algorithm acceleration.

Week 9 and Beyond

Recommended order:

1. Decision Trees
2. Random Forests
3. K-Means
4. SVM
5. PCA
6. Cross-validation and search
7. Advanced C optimization

Learning Checkpoints

Use these checkpoints to make sure you are learning deeply, not just adding files.

After Linear Regression, you should be able to explain:

- ? What gradient descent is doing.
- ? Why feature scaling matters.
- ? How learning rate affects convergence.
- ? Why loss can diverge.
- ? How the implementation stores weights and bias.

After Logistic Regression, you should be able to explain:

- ? Why sigmoid needs numerical care.
- ? What cross-entropy measures.
- ? How probabilities become class labels.
- ? Why classification metrics differ from regression metrics.

After Decision Trees, you should be able to explain:

- ? How Gini and entropy measure impurity.
- ? Why greedy splitting works reasonably well.
- ? Why trees overfit.
- ? How max depth affects bias and variance.

After C kernels, you should be able to explain:

- ? Why Python loops are slow.
- ? How contiguous memory helps performance.
- ? How ctypes passes data.
- ? Why benchmarks must compare equal workloads.

What Not To Do Early

Avoid these until the basics are stable:

- ? Do not start with neural networks.
- ? Do not add GPU support.
- ? Do not use NumPy to power core implementations.
- ? Do not write C before the Python version is correct.
- ? Do not implement many algorithms without tests.
- ? Do not optimize code that has not been benchmarked.
- ? Do not chase every feature from scikit-learn at once.

Current Next Step

The immediate next step is:

Create the repository skeleton and first utility modules.

Start with Phase 0 and Phase 1:

1. Create folders.
2. Create `mlscratch` package.
3. Add import tests.
4. Add validation helpers.
5. Add regression metrics.
6. Add train/test split.

Once those are done, the project is ready for the first real model: Linear Regression.